Summary

The task is to build a classification model that can accurately predict the vegetation condition of each land parcel on Wadjuk Noongar country into the three categories, which are 'Healthy', 'Degraded' and 'AtRisk'. Understanding the condition of vegetation is essential for maintaining biodiversity, managing fire risk, guiding cultural burns, and protecting culturally significant areas. The given dataset is 'Assignment2025S2.sqlite', where each record contains various indicators of environmental condition, such as vegetation health, soil properties, moisture availability, proximity to water, and signs of human or invasive species disturbance. There are a total of 5500 samples across two tables, which are 'train' and 'test'.

Before data classification, data preparation was conducted, including the identification and removal of irrelevant attributes, the detection and handling of missing entries, the detection and handling of duplicates, the selection of suitable data types for attributes and data transformation. For data classification, 4 models were used, which are K-NN, Decision tree, Naive Bayes and SVM. The Models were evaluated by using a confusion matrix, accuracy, precision, recall and F1-score. The Naive Bayes model and the SVM model were considered best for the task, achieving 80% estimated accuracy and 88% estimated accuracy, respectively.

Methodology

1. Creation of test/train splits

Initially, the data needs to be split into training data and testing data. The given dataset 'Assignment2025S2.sqlite' is composed of two tables, which are the train table and the test table. The given table 'train' was separated into training data and testing data, where the training data constituted 80% of the table 'train', whereas the testing data constituted 20%. And the given table 'test', which is without an actual class label, was considered as unseen data.

The task is to train a machine learning model that will predict the vegetation condition of each land parcel on Wadjuk Noongar country into the three categories, which are 'Healthy', 'Degraded' and 'AtRisk'. These three categories are the target, which is denoted as y, and the remainder of the attributes are the inputs, which are denoted as X.

The training data was divided into X_train and y_train, and the testing data was divided into X_test and y_test. The unseen data was regarded as X_unseen. In data preparation, information leaking from testing data to training data must be taken into account. Therefore, the X_train was used to determine how the data is pre-processed and scaled.

2. Data Preparation

Data preparation is required for a machine learning model to perform classification because raw data is not ideal for them. It's carried out in four phases in this task, which are 'Identification and removal of irrelevant attributes', 'Detection and handling of missing entries', 'Detection and handling of duplicates' and 'Data transformation'.

2.1. Identification and removal of irrelevant attributes

The index attribute didn't contain useful information as it simply represented the row number. It wasn't beneficial for the machine learning model to classify the vegetation condition, thus it was removed. The same procedure was applied to the X_test and the X_unseen.

2.2. Detection and handling of missing entries

Missing entries need to be detected and handled. Attributes, which have a large number of missing entries, are not valuable because information is not provided by them. Therefore, they need to be removed. However, Attributes with small amounts of missing entries can be imputed using either the mean or the mode of the remaining data, depending on the data type.

2.2.1. Detection of missing entries

As a result of examining the X_train, missing values were detected in three columns, which were 'Greenness_Index', 'Days_Since_Burn' and 'Latitude'. The fraction of missing values of the column 'Greenness_Index' was 0.02575, the fraction of missing values of the column 'Days_Since_Burn' was 0.21400, and the fraction of missing values of the column 'Latitude' was 0.57900. The column 'Latitude' was removed because it had excessive missing values, such as more than 50%. Furthermore, it was removed in the X_test and the X_unseen.

2.2.2. Selection of suitable data types for attributes

In the case of the attributes 'Greenness_Index' and 'Days_Since_Burn', these attributes had small amounts of missing values; thus, they were imputed. Before the imputation is conducted, the data types of the attributes need to be assessed. Therefore, at this stage, the attributes of the X_train were examined to select suitable data types for the attributes.

Since the irrelevant attribute 'index' was removed, there are 29 attributes in the X_train. The examination of the X_train indicated that there are 3 distinct data types, which are 'float64', 'int64' and 'object'. The X_train contains 20 attributes of the type 'float64', 4 attributes of the type 'int64' and 5 attributes of the type 'object'. After verifying whether the attributes, which have the type 'int64' and 'object', are drawn from a restricted set, they will be considered to be converted into the type 'category'.

The attributes 'Cultural_Burn', 'Flood_Risk_Zone', 'Invasive_Species_Presence' and 'Litter_Presence' have the type 'int64'. They are all drawn from a restricted set, especially binary, which contains only 0 and 1 values. Therefore, they were converted into the type 'category'.

The attributes 'Soil_Type', 'Burn_Season', 'Surface_Water_Presence', 'Landform', and 'Human_Disturbance' have the type 'object'. The attributes have the values below.

The attribute	Values
'Soil_Type'	'Silty', 'Loam', 'Peaty', 'Clay', 'Sandy', 'Gravel'
'Burn_Season'	'Djeran', 'Bunuru', 'Makuru', 'Birak', 'Djilba', 'Kambarang'
'Surface_Water_Presence'	'Dry', 'Seasonal', 'Permanent'
'Landform'	'Hill', 'Flat', 'Valley', 'Ridge', 'Dune'
'Human_Disturbance'	'Low', 'Medium', 'High'

They are all drawn from a restricted set. Therefore, they were converted into the type 'category'. The data conversion of the attributes was applied to the X_test and the X_unseen.

2.2.3. Imputation of missing entries

As a result of the examination of the X_train, the attributes 'Greenness_Index' and 'Days_Since_Burn', which have missing entries, have the type 'float64'. Since they had numerical values, the missing entries were imputed using the mean value of the remaining data of each attribute. These 2 mean values were used to impute the missing values of the attributes in the X_test and the X_unseen. As mentioned above, information leaking from testing data to training data must be taken into account. Therefore, the X_train was used to determine how the data was pre-processed and scaled in either the X_train, the X_test and the X_unseen.

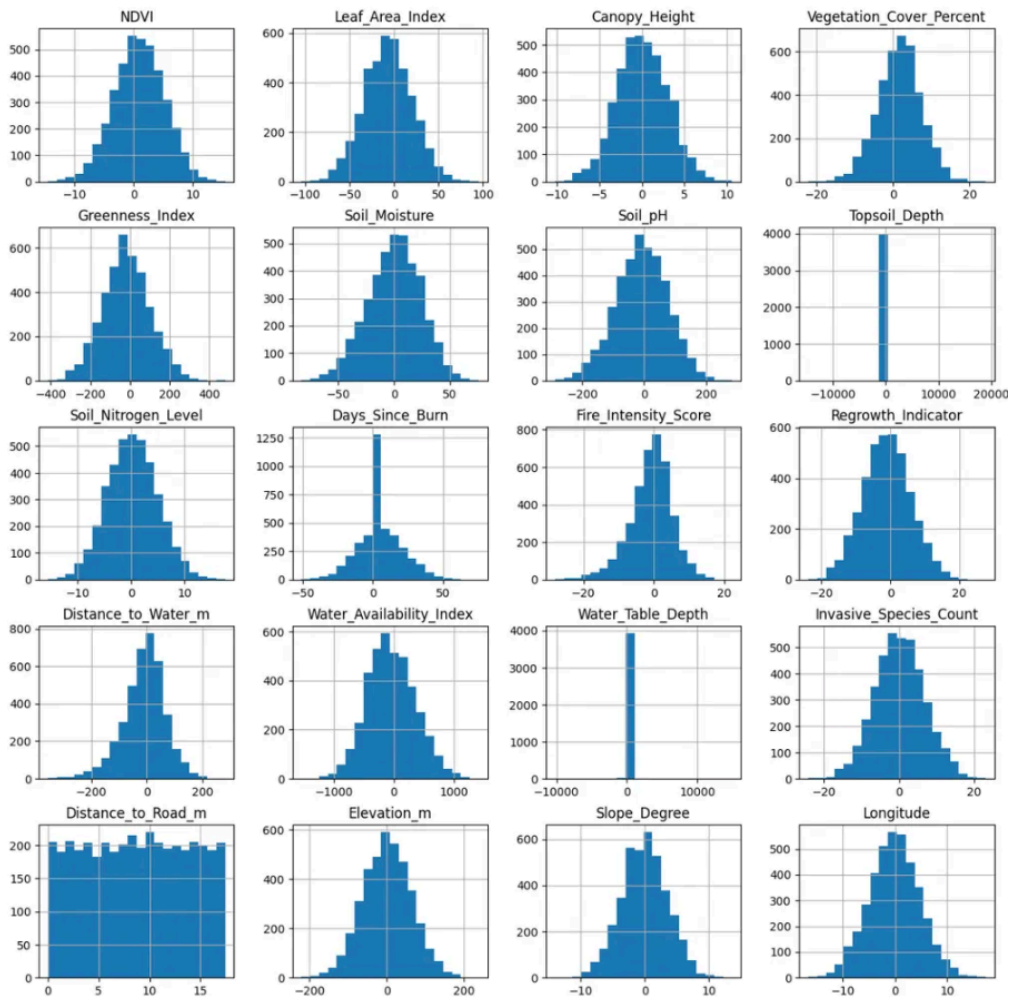
2.3. Detection and handling of duplicates

Duplicate detection is important because duplicates bias the model. Therefore, both instances and attributes need to be assessed to determine whether they have duplicates. 37 rows were detected as duplicates out of 4000 rows in the X_train. The fraction of the duplicated rows is only 0.00925. Hence, they were not processed as it wasn't expected that they would cause a significant impact on the performance of the machine learning model. No duplicate attributes were detected out of 29 attributes in the X_train. Therefore, no further action was required.

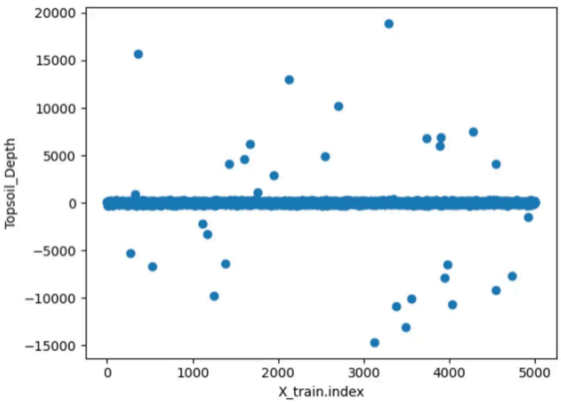
2.4. Data transformation

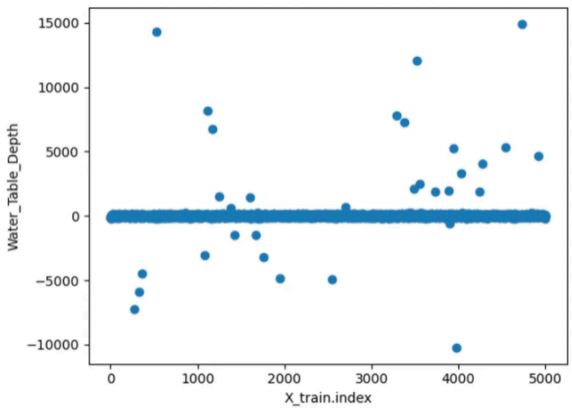
2.4.1. Scaling/Standardisation

Attributes with different scales or distributions can cause bias in Data Mining applications. Therefore, scaling or standardisation is required. There are 20 distributions of the numerical attributes below.



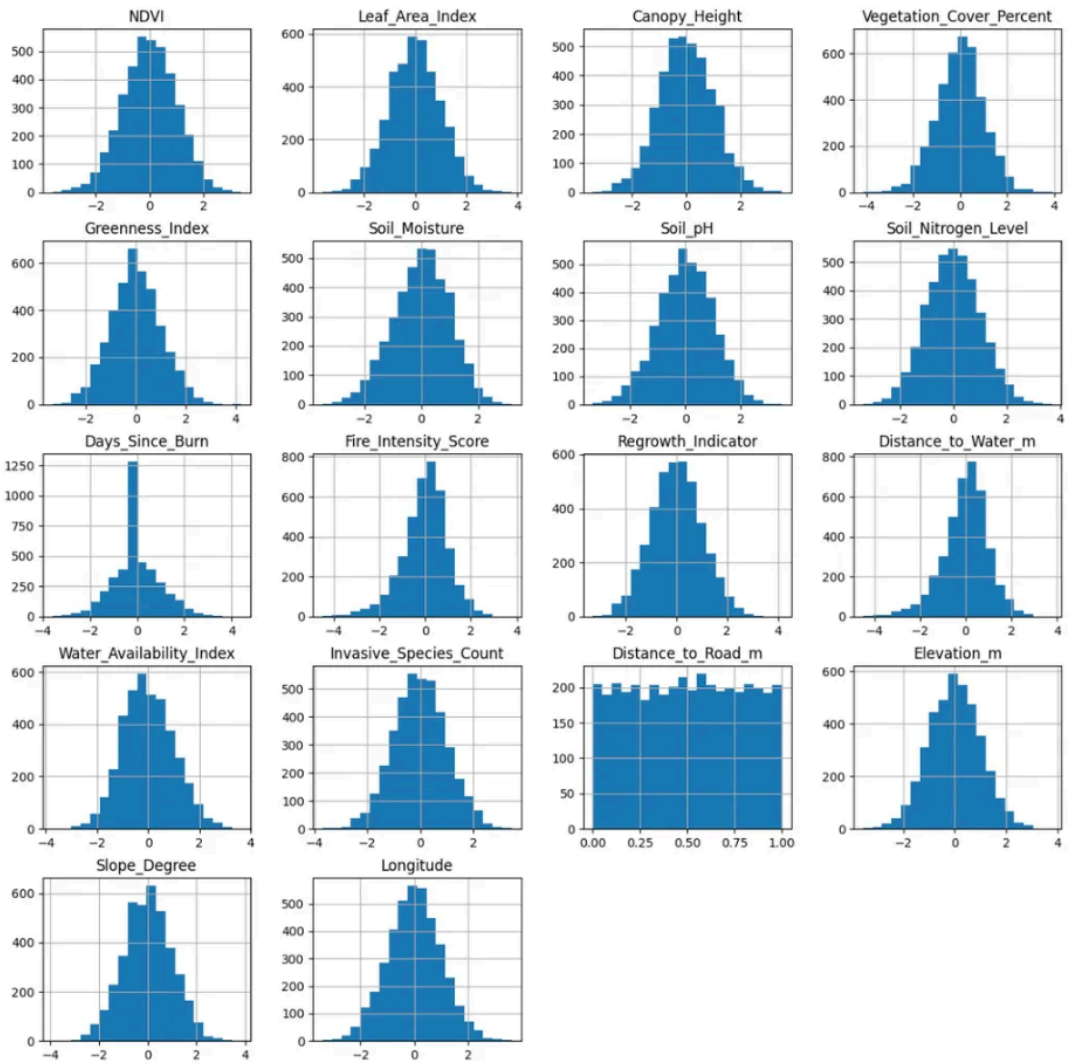
These distributions can be distinguished as a normal distribution and a uniform distribution, depending on the form of the distribution. Since the distributions of the columns 'Topsoil_Depth' and 'Water_Table_Depth' slightly deviated from the norm, further investigation was required to determine whether this was caused by the fact that these attributes didn't have much variance.





From the above scatter plots of the two attributes, only a small number of outliers were detected. Furthermore, when it comes to inliers, the value of the measurement did not change significantly. In other words, the two attributes had very little variance, which indicated that they didn't distinguish between the different data points. Therefore, both attributes were removed. They were also removed in the X_test and the X_unseen.

18 numerical attributes remained, the attribute 'Distance_to_Road_m' has a uniform distribution, and the rest of the attributes have normal distributions. Therefore, min-max scaling was applied to the attribute 'Distance_to_Road_m' and z-score normalisation(standardisation) was applied to the rest attributes. The same procedure was applied to the X_test and X_unseen. The distributions of the numerical attributes after the scaling are as shown below.



2.4.2. One-hot encoding

One-hot encoding is needed for feeding categorical data to the machine learning models. There are 9 categorical attributes in the X_train, which are 'Soil_Type', 'Burn_Season', 'Surface_Water_Presence', 'Landform', 'Human_Disturbance', 'Cultural_Burn', 'Flood_Risk_Zone', 'Invasive_Species_Presence' and 'Litter_Presence'.

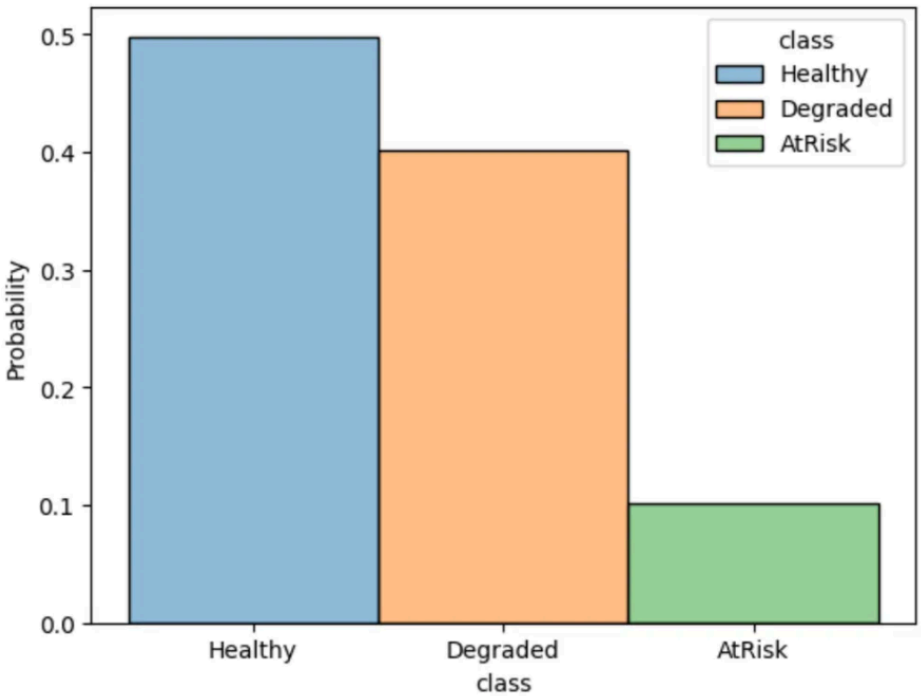
The attributes 'Cultural_Burn', 'Flood_Risk_Zone', 'Invasive_Species_Presence', and 'Litter_Presence' are binary, as mentioned above, which indicates one-hot encoding is not required for these attributes. Therefore, one-hot encoding was applied to the rest attributes. The same procedure was applied to the X_test and the X_unseen.

3. Data Classification

3.1. Dimensionality Reduction by using PCA

After applying one-hot encoding, the number of attributes became 40. High dimensionality leads to low contrast, which indicates that the differences between data points become less relevant. Therefore, the machine learning models perform inadequately at high dimensions. Therefore, PCA was used to reduce dimensionality and determine how many components were required to explain 90% of the variance. After applying PCA, 16 components were found. Therefore, the number of dimensions was reduced from 40 to 16 through PCA. PCA was also applied to the X_test and the X_unseen.

3.2. Addressing class imbalance by using SMOTE



The original labelled data is not equally distributed between the three classes, as shown above. It's problematic because the class 'AtRisk' is rare, and therefore misclassification is likely to occur. However, it's critical to identify. Hence, the class imbalance needs to be addressed. SMOTE was used to address this issue because it creates synthetic data by making a linear combination of existing data, which generates high-quality training datasets (Azad et al., 2022). Furthermore, cross-validation with the stratified sampling method was used to address this issue; therefore, every cross-validation set has the minor class, which is 'AtRisk'. After applying SMOTE to the X_train and the y_train, each class acquired 1983 samples. These datasets are considered as X_train_smote and y_train_smote.

3.3. Model training and tuning

Since data preparation and dimensionality reduction were conducted, and class imbalance was also addressed, the machine learning models are prepared to be trained and tuned. According to the given condition, K-NN, Decision Tree and Naive Bayes were used as the machine learning models, and additionally, SVM was used. The reason why SVM was used is that it's effective in high-dimensional spaces and uses a subset of training points in the decision function. Therefore, it is memory efficient.

3.3.1. Tuning model hyperparameters by using cross-validation

Each classifier has hyperparameters to tune. Values that produced the best average performance were selected by using cross-validation. The reason why cross-validation was used is to ensure the performance of the model wasn't dependent on a specific choice of which data points were selected for model-building or validation. Therefore, the model was protected from the random chance that the performance of the model was conditioned by a validation set. Specifically, stratified 10-fold cross-validation using a random state of 42 was used. Out of 10 iterations, the average accuracy was considered the true accuracy of the model. A standard deviation was considered for the confidence in the performance of the model.

The hyperparameters of each model and their values over the grid search were selected to find the best parameters. When the grid search was conducted, the X_train_smote and the y_train_smote were used. The best parameters were determined after conducting the grid search. Once they were determined, they were used to make a new classifier, which would become the best model with the optimum parameters. And then the best model with the optimum parameters was retrained using all training data.

As part of the cross-validation, the average accuracy and standard deviation across the different subsets were investigated. Based on this information, an accuracy on the testing data was expected. And then eventually the best model was tested on the testing data and evaluated by looking at several classification performance metrics, which are accuracy, precision, recall, and F1-score.

3.3.2. K-NN

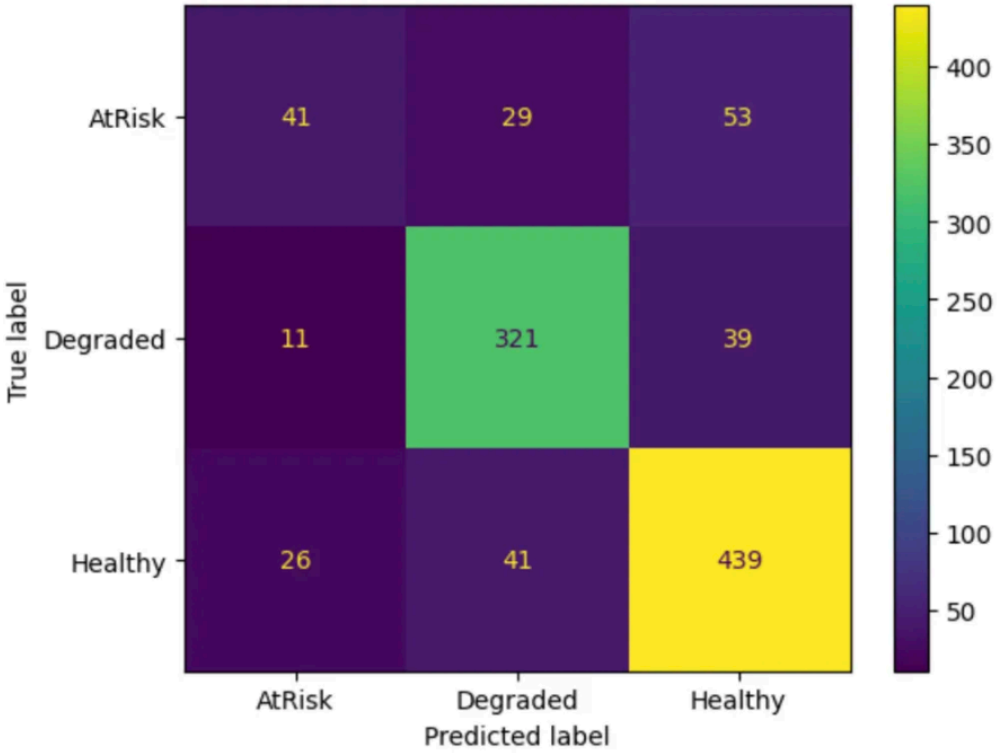
3.3.2.1. Tuning hyperparameters

'Weights' and 'n_neighbours' were selected as hyperparameters of the K-NN model. The choice of n_neighbours is important because nearest neighbours mean the smallest distances, which decide the most similar data points to the data that needs to be labelled. The values of 'n_neighbours' were set to [1, 3, 7, 11, 17, 21]. Furthermore, the choice of distance metric is also important because it calculates data similarity and distances. The distance can be weighted, which indicates that data points that are closer to the data points that need to be labelled can obtain more weight than those further away. The values of 'weight' were set to ['uniform', 'distance']. 'Uniform' weights all points in each neighbourhood equally, and 'distance' weights points by the inverse of their distance. In this case, closer neighbours of a query point will have a greater influence than neighbours that are further away. As a result of the cross-validation with the hyperparameters, the best parameter of 'n_neighbours' was 1 and the best parameter of 'weight' was 'uniform'.

The best K-NN model using the optimum parameters was retrained on all the training data. As a result of the cross-validation, the average accuracy was 0.803 and the standard deviation was 0.023, which indicates that the best K-NN model had an accuracy of 80% with 2% variance, demonstrating that the performance of the best K-NN model is stable. Therefore, it was expected to have an accuracy of 80% on the testing data. As a result of the classification on the testing data, it had an accuracy of 0.801, which means 80% accuracy. The result matched the average accuracy of the cross-validation.

3.3.2.2. Performance metric

Since the predicted set of labels was obtained, the performance of the best K-NN model could be evaluated with a confusion matrix and some metrics, which were accuracy, precision, recall and F1-score.



The matrix above shows the true label and the predicted label for each sample. The model was evaluated by using a confusion matrix for each class, which is shown below.

Confusion matrix

	Actual 1	**Actual 0**
Predicted 1	True Positive	False Positive
Predicted 0	False Negative	True Negative

From this, the model could be evaluated with the following metrics :

- Accuracy: Fraction of test instances that are correctly labelled: (TP+TN) / (TP+FP+FN+TN)
- Precision: Fraction reported positives that are correct: TP / (TP+FP)
- Recall: Fraction of positives that are reported: TP / (TP+FN)
- F1 Score: The harmonic mean of precision and recall, which provides a balanced measure of the model's performance : (2*Precision*Recall) / (Precision+Recall)

3 confusion matrices were produced for each class. With them, the model could be evaluated in terms of classification performance for each class. Since the class imbalance was problematic, it was expected that the minor class ‘AtRisk’ wouldn’t be misclassified.

Confusion matrix for the class ‘AtRisk’

	Actual 1	Actual 0
Predict 1	41	37
Predict 0	82	840

Confusion matrix for the class ’Degraded’

	Actual 1	Actual 0
Predict 1	321	70
Predict 0	50	559

Confusion matrix for the class ‘Healthy’

	Actual 1	Actual 0
Predict 1	439	92
Predict 0	67	402

Performance metrics for each class

	Accuracy	Precision	Recall	F1-score
AtRisk	0.88	0.53	0.33	0.41
Degraded	0.88	0.82	0.87	0.84
Healthy	0.84	0.83	0.87	0.85

When it comes to the class ‘AtRisk’, the model has an accuracy of 88%. However, the other metrics are considerably low. It has a precision of 53%, a recall of 33% and an F1-score of 41%. When it comes to the class ‘Degraded’, the model has an accuracy of 88%, a precision of 82%, a recall of 87% and an F1-score of 84%, which indicates the model performed well overall. When it comes to the class ‘Healthy’, the model has an accuracy of 84%, a precision of 83%, a recall of 87% and an F1-score of 85%, which indicates the model performed well overall.

In summary, the best K-NN model performs effectively on the common classes, which are ‘Healthy’ and ‘Degraded’. However, it performs inadequately on the rare class ‘AtRisk’. Therefore, the best K-NN model may not perform successfully on the unseen data. For example, it may predict the classes that are not ‘AtRisk’ appropriately; however, it may not predict the class ‘AtRisk’ properly.

3.3.3. Decision Tree

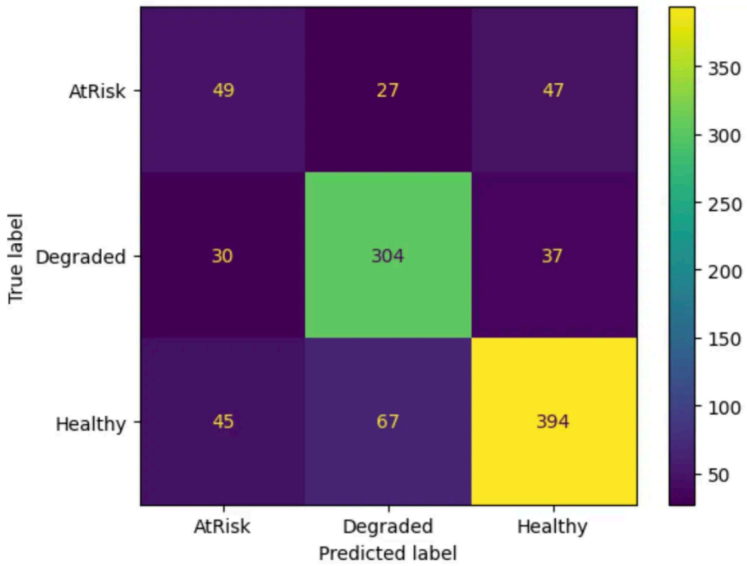
3.3.3.1. Tuning hyperparameters

‘Criterion’, ’max_depth’ and ‘min_samples_split' were selected as hyperparameters of the Decision Tree model. The hyperparameter ‘criterion’ is important because it measures the quality of the split. The values of ‘criterion’ were set to [‘gini’, ‘entropy’]. The criterion ‘gini’ uses the gini index for the criterion of the separation of the different classes, which is the measure of diversity. The other criterion, ‘entropy’, is the measure of disorder or uncertainty. The hyperparameter ‘max_depth’ is important because if ‘max_depth’ is ‘None’, then nodes are expanded until all leaves are pure or until all leaves contain less than the value of the hyperparameter ‘min_samples_split’. The values of ‘max_depth’ were set to [‘None’, 5, 10]. The hyperparameter ‘min_samples_split’ is crucial because it gives the model a stopping criterion. It prevents further splits of an attribute when the number of data points in the split is less than the value of ‘min_samples_split’. The values of ‘min_samples_split’ were set to [2, 5, 10].

As a result of the cross-validation with the hyperparameters, the best parameter of ’criterion’ was ‘entropy’, the best parameter of ‘max_depth’ was ‘None’ and the best parameter of ‘min_samples_split’ was 5. The best model using the optimum parameters was retrained using all the training data. As a result of the cross-validation, the average accuracy was 0.756 and the standard deviation was 0.015, which indicates that the best Decision Tree model had an accuracy of 76% with 1.5% variance, demonstrating that the performance of the model is stable. Therefore, it was expected to have an accuracy of 76% on the testing data. As a result of the classification of the best Decision Tree model on the testing data, it had an accuracy of 0.747, which means 75% accuracy. The results were within 2% error of the average accuracy of the cross-validation.

3.3.3.2. Performance metric

Since the predicted set of labels was obtained, the performance of the best Decision Tree model could be evaluated with a confusion matrix and some metrics, which were accuracy, precision, recall and F1-score.



3 confusion matrices were produced for each class below. With them, the model could be evaluated in terms of classification performance for each class. Since the class imbalance was problematic, it was expected that the minor class 'AtRisk' wouldn't be misclassified.

Confusion matrix for the class 'AtRisk'

	Actual 1	**Actual 0**
Predict 1	49	75
Predict 0	74	802

Confusion matrix for the class 'Degraded'

	Actual 1	**Actual 0**
Predict 1	304	94
Predict 0	67	535

Confusion matrix for the class 'Healthy'

	Actual 1	**Actual 0**
Predict 1	394	84
Predict 0	112	410

Performance metrics for each class

	Accuracy	**Precision**	**Recall**	**F1-score**
AtRisk	0.85	0.40	0.40	0.40
Degraded	0.84	0.76	0.82	0.79
Healthy	0.80	0.82	0.78	0.80

When it comes to the class 'AtRisk', the model has an accuracy of 85%. However, the other metrics are considerably low. It has a precision of 40%, a recall of 40% and an F1-score of 40%. When it comes to the class 'Degraded', the model has an accuracy of 84%, a precision of 76%, a recall of 82% and an F1-score of 79%, which indicates the model performed well overall. When it comes to the class 'Healthy', the model has an accuracy of 80%, a precision of 82%, a recall of 78% and an F1-score of 80%, which indicates the model performed well overall.

In summary, the best Decision Tree model performs effectively on the common classes, which are 'Healthy' and 'Degraded'. However, it performs inadequately on the rare class 'AtRisk'. Therefore, the best Decision Tree model may not perform successfully on the unseen data, such as the best K-NN model. For example, it may predict the classes that are not 'AtRisk' appropriately; however, it may not predict the class 'AtRisk' properly.

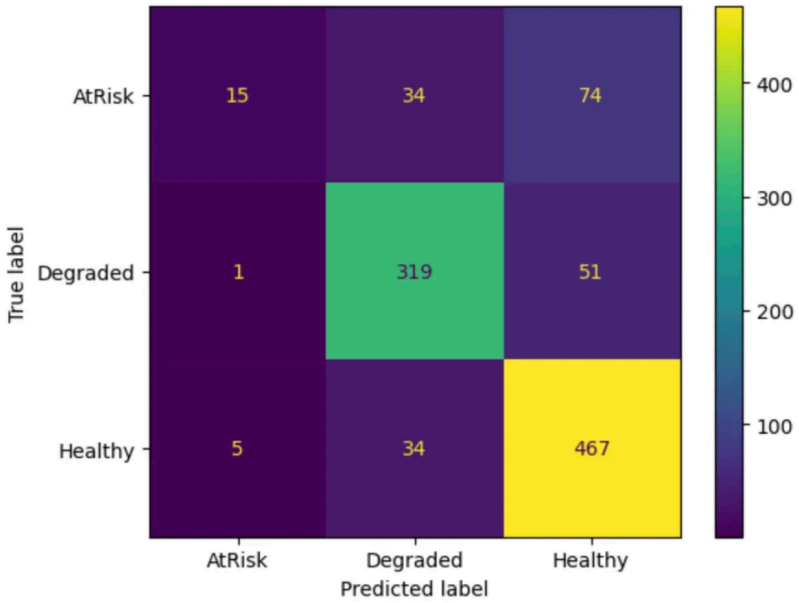
3.3.4. Naive Bayes

3.3.4.1. Tuning hyperparameters

Naive Bayes classifier doesn't have any parameters that are worth tuning; however, cross-validation can still be conducted. After applying the cross-validation, the best Naive Base model was retrained using all the training data. As a result of the cross-validation, the average accuracy was 0.819 and the standard deviation was 0.013, which indicates that the best Naive Bayes model had an accuracy of 82% with 1.3% variance, demonstrating that the performance of the best Naive Bayes model is stable. Therefore, it was expected to have an accuracy of 82% on the testing data. As a result of the classification on the testing data, it had an accuracy of 0.801, which means 80% accuracy. The results were within 2% error of the average accuracy of the cross-validation.

3.3.4.2 Performance metric

Since the predicted set of labels was obtained, the performance of the best Naive Bayes model could be evaluated with a confusion matrix and some metrics, which were accuracy, precision, recall and F1-score.



3 confusion matrices were produced for each class below. With them, the model could be evaluated in terms of classification performance for each class. Since the class imbalance was problematic, it was expected that the minor class 'AtRisk' wouldn't be misclassified.

Confusion matrix for the class 'AtRisk'

	Actual 1	**Actual 0**
Predict 1	15	6
Predict 0	108	871

Confusion matrix for the class 'Degraded'

	Actual 1	**Actual 0**
Predict 1	319	68
Predict 0	52	561

Confusion matrix for the class 'Healthy'

	Actual 1	**Actual 0**
Predict 1	467	125
Predict 0	39	369

Performance metrics for each class

	Accuracy	**Precision**	**Recall**	**F1-score**
AtRisk	0.89	0.71	0.12	0.21
Degraded	0.88	0.82	0.86	0.84
Healthy	0.84	0.79	0.92	0.85

When it comes to the class ‘AtRisk’, the model has an accuracy of 89% and a precision of 71%. However, it has a recall of 12% and an F1-score of 21%. The two metrics, which are precision and recall, have a substantial difference in performance. Precision is fairly high, which indicates that once the model predicts the class as ‘AtRisk’, the prediction is accurate. However, recall is considerably low, which demonstrates that among the actual class ‘AtRisk’, the fraction of the class ‘AtRisk’ that are reported is low. However, precision is more practical to look at the confusion matrix because, in practice, the truth of the unseen data points is generally unknown.

When it comes to the class ‘Degraded’, the model has an accuracy of 88%, a precision of 82%, a recall of 86% and an F1-score of 84%, which indicates the model performed well overall. When it comes to the class ‘Healthy’, the model has an accuracy of 84%, a precision of 79%, a recall of 92% and an F1-score of 85%, which indicates the model performed well overall. Especially, the recall is fairly high, which indicates that among the actual class ‘healthy’, the fraction of the class ‘healthy’ that are reported is high.

In summary, the best Naive Bayes model performs effectively on the common classes, which are ‘Healthy’ and ‘Degraded’. However, in the rare class ‘AtRisk’, it has a low recall and F1-score. However, it has high precision, which indicates that once the model predicts the class as ‘AtRisk’, the prediction is fairly accurate. Therefore, in comparison, the best Naive Bayes model performs more appropriately than the best K-NN model and the best Decision Tree model.

3.3.5. SVM

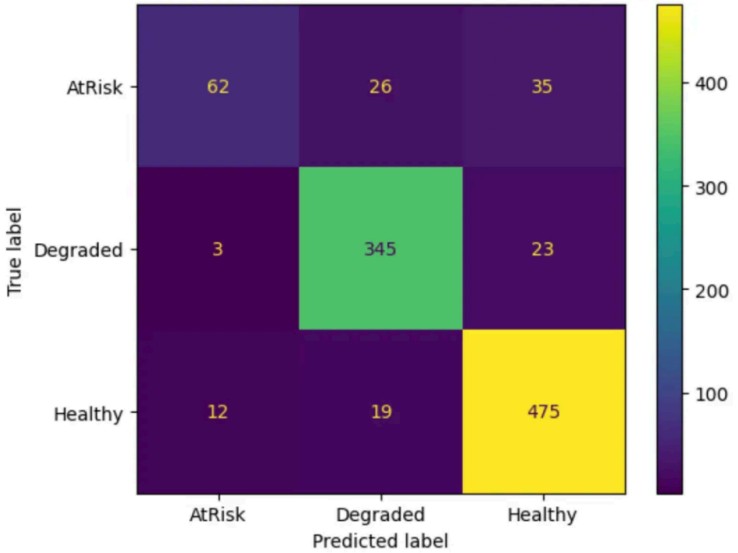
3.3.5.1. Tuning hyperparameters

‘c’, ‘kernel’ and ‘gamma' were selected as hyperparameters of the SVM model. The hyperparameter ‘c’ is essential because it is a regularisation parameter. The values of ‘c’ were set to [0.1, 1, 10]. The hyperparameter ‘kernel’ is crucial because it specifies the kernel type to be used in the algorithm. The values of ‘kernel’ were set to [‘linear’, ‘rbf’]. The hyperparameter ‘gamma’ is also important because it’s the kernel coefficient for ‘rbf’, which is the value of ‘kernel’. The values of ‘gamma’ were set to [‘scale’, ‘auto’]. As a result of the cross-validation with the hyperparameters, the best parameter of ’c’ was 10, the best parameter of ‘gamma’ was ‘scale’, and the best parameter of ‘kernel’ was ‘rbf’. The best model was retrained using the optimum parameters and all the training data.

As a result of the cross-validation, the average accuracy was 0.884 and the standard deviation was 0.012, which indicates that the best SVM model had an accuracy of 88% with 1.2% variance, demonstrating that the performance of the best SVM model is stable. Therefore, it was expected to have an accuracy of 88% on the testing data. As a result of the classification of the best SVM model on the testing data, it had an accuracy of 0.882, which means 88% accuracy. The result matched the average accuracy of the cross-validation.

3.3.5.2. Performance metric

Since the predicted set of labels was obtained, the performance of the best SVM model could be evaluated with a confusion matrix and some metrics, which were accuracy, precision, recall and F1-score.



3 confusion matrices were produced for each class below. With them, the model could be evaluated in terms of classification performance for each class. Since the class imbalance was problematic, it was expected that the minor class 'AtRisk' wouldn't be misclassified.

Confusion matrix for the class 'AtRisk'

	Actual 1	Actual 0
Predict 1	62	15
Predict 0	61	862

Confusion matrix for the class 'Degraded'

	Actual 1	Actual 0
Predict 1	345	45
Predict 0	26	584

Confusion matrix for the class 'Healthy'

	Actual 1	Actual 0
Predict 1	475	58
Predict 0	31	436

Performance metrics for each class

	Accuracy	Precision	Recall	F1-score
AtRisk	0.92	0.81	0.50	0.62
Degraded	0.93	0.88	0.93	0.91
Healthy	0.91	0.89	0.94	0.91

When it comes to the class 'AtRisk', the model has an accuracy of 92% and a precision of 81%. However, it has a recall of 50% and an F1-score of 62%. The two metrics, which are precision and recall, have a substantial difference in performance. As mentioned above, precision is a more practical way to look at the confusion matrix. In comparison with the best Naive Bayes model, which has a precision of 71% and a recall of 12% whereas the best SVM model has higher precision and recall, which indicates that when the best SVM model predicts the class 'AtRisk', the prediction is more accurate than the best Naive Bayes model as well as it's more likely to report the class as 'AtRisk' among the actual class 'AtRisk' than the best Naive Bayes model.

When it comes to the class 'Degraded', the model has an accuracy of 93%, a precision of 88%, a recall of 93% and an F1-score of 91%, which indicates the model performed well overall. When it comes to the class 'Healthy', the model has an accuracy of 91%, a precision of 89%, a recall of 94% and an F1-score of 91%, which indicates the model performed well overall. Especially, the recall is fairly high, which indicates that among the actual 'healthy', the fraction of 'healthy' that are reported is high.

In summary, the best SVM model performs effectively on the common classes, which are 'Healthy' and 'Degraded'. For the rare class 'AtRisk', even though it has a low recall and F1-score, it has high precision, which indicates that once the model predicts the class as 'AtRisk', the prediction is fairly accurate. Furthermore, it has higher precision and recall than the best Naive Bayes model. Therefore, in comparison, the best SVM model performs more appropriately than the best Naive Bayes model, which indicates it performs the best among the 4 models.

3.4. Model comparison

The criteria for comparing 4 models are the accuracy on the testing data and the precision on the rare class 'AtRisk'. The reason why accuracy was selected is that, based on that accuracy, the model is expected to achieve at least that accuracy on the unseen data. And the reason why the precision on the class 'AtRisk' was selected is that every model performs successfully on the common classes 'Degraded' and 'Healthy', and it's critical to identify the rare class 'AtRisk'.

When comparing the 4 models by accuracy, the best SVM model is the best among the 4 models.

The model	The accuracy
The best K-NN model	80%
The best Decision Tree model	75%
The best Naive Bayes model	80%
The best SVM model	88%

When comparing the 4 models by precision on the class 'AtRisk', the best SVM model has the highest precision, and the best Naive Bayes model has the second-highest precision.

The model	The precision on the class 'AtRisk'
The best K-NN model	53%
The best Decision Tree model	40%
The best Naive Bayes model	71%
The best SVM model	81%

Since the best SVM model has the highest accuracy on the testing data and the highest precision on the class 'AtRisk', it's the best model among the 4 models without doubt. The other best model is the best Naive Bayes model because it has the second-highest accuracy on the testing data and the second-highest precision on the class 'AtRisk'. Therefore, the best 2 models are the best Naive Bayes model and the best SVM model.

3.5. Prediction on the unseen dataset

At this stage, it is appropriate for the best 2 models to predict missing class labels on the unseen dataset. The estimated accuracy of the best Naive Bayes model is 80% and the estimated accuracy of the best SVM model is 88%.

The best model	Estimated accuracy
The best Naive Bayes model	80%
The best SVM model	88%

In the case of the best Naive Bayes model, it has an average accuracy of 82% with 1.3% variance with cross-validation, which indicates the performance of the model is stable. Most importantly, it has an accuracy of 80% on the testing data. Therefore, it's expected to achieve at least 80% accuracy on the unseen data.

In the case of the best SVM model, it has an average accuracy of 88% with 1.2% variance with cross-validation, which indicates the performance of the model is stable. Most importantly, it has an accuracy of 88% on the testing data. Therefore, it's expected to achieve at least 88% accuracy on the unseen data.

The classification table was produced as the Answers_23549670.csv file. The first column is the index corresponding to the 'test' table, the second and third columns are the predicted class labels. The second column, 'Predict 1', was populated with the result of the classification of the best Naive Bayes model. The third column, 'Predict 2,' was populated with the result of the classification of the best SVM model.

Conclusion

The best K-NN model has an 80% accuracy, the best Decision Tree model has a 75% accuracy, the best Naive Bayes model has an 80% accuracy, and the best SVM model has an 88% accuracy. However, the models need to be evaluated by several classification performance metrics such as accuracy, precision, recall and F1-score because accuracy doesn't explain every aspect of the performance of the model, especially when there is a class imbalance issue. When it comes to the classification of the common classes, which are 'Degraded' and 'Healthy', all 4 models perform appropriately on the 4 metrics. However, when it comes to the classification of the rare class 'AtRisk', even though the accuracy of the 4 models is fairly high, it doesn't demonstrate strong performance in predicting the class 'AtRisk', which indicates that other metrics need to be used.

The best K-NN model and the best Decision Tree model have considerably low precision, recall and F1-score, which indicates the models may not perform appropriately on the unseen data. By contrast, the precision of the Naive Bayes model and the SVM model was relatively high, which demonstrates these 2 models can predict the class 'AtRisk' fairly accurately. However, the recall of these 2 models was substantially low as well, which indicates that among the actual class 'AtRisk', the fraction of the class 'AtRisk' that are reported is low. In this task, the rare class 'AtRisk' is critical to identify; therefore, the accuracy of the model on the testing data and the precision of the model on the class 'AtRisk' were selected to evaluate and compare the models.

It's important to note that this task has several limitations. As mentioned above, every 4 models have a low recall and F1-score. To address this, a recall-optimised machine learning framework that strongly penalises misclassification of a rare class can be used. (Jaroy, E. G., et al., 2024) And SMOTE is a good sampling technique to make new samples for minor classes; however, some of the synthetic training data may be the same as the samples of common classes, which could be considered another limitation. To address this, SMOTE-MRS could be used. The SMOTE-MRS model mitigates the issue of imbalanced datasets by combining the techniques of SMOTE, K-Means clustering, and random oversampling. (Saputra, Ekty. Chaerul. D., et al., 2024)

Also, during the feature selection process, the 2 attributes, which are 'Topsoil_Depth' and 'Water_Table_Depth', were removed; however, instead of the removal of the attributes, the small amount of the outliers could be removed, and then the variance of the inliers could be investigated. Lastly, there are some incorrect data, such as negative values, which are not supposed to be found in the attributes such as 'Canopy_Height', 'Vegetation_Cover_Percent', and 'Days_Since_Burn'. Remeasure could potentially be performed, but the probability of occurrence is minimal.

References

Aggarwal, C. C. (2015). Data Mining: The Textbook. Springer.

Azad, C., Bhushan, B., Sharma, R., Shankar, A., Singh, K. K., & Khamparia, A. (2022). "Prediction model using SMOTE, genetic algorithm and decision tree (PMSGD) for classification of diabetes mellitus." Multimedia systems 28(4): 1289-1307.

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.T.html

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html#sklearn.metrics.precision_recall_fscore_support

https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html

https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html#sklearn.svm.SVC

https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

https://scikit-learn.org/stable/modules/naive_bayes.html

https://scikit-learn.org/stable/modules/svm.html

Jaroy, E. G., Risa, G. T., Farstad, I. N., Emblem, R. & Ougland, R. (2024). "A recall-optimised machine learning framework for small data improves risk stratification for Hirschsprung's disease." Informatics in medicine unlocked 48: 101530.

Saputra, Ekty. Chaerul. D., Sunat, K., Ratnaningsih, T. (2024). "SMOTE-MRS: A Novel SMOTE-Multiresolution Sampling Technique for Imbalanced Distribution to Improve Prediction of Anemia." IEEE access 12: 154675-154699.